

Plan de Rapport : Conception d'une Architecture Processeur 16-bits

Informations sur le document :

- **Auteurs:** Houssam Bendriouich, Kyllian Siewe Tiague, Wassim Laarbi
- **Date de finalisation :** 25 mai 2026 00:00

I - INTRODUCTION : Objectifs et contexte pédagogique.....	1
1.1 Contexte pédagogique.....	1
1.2 Objectif du projet.....	1
II- ARCHITECTURE : Architecture et matériels.....	2
2.1. Les Registres.....	2
2.2. La Mémoire Centrale (RAM).....	2
2.3. L'Unité Arithmétique et Logique (UAL).....	2
2.4. Le Chemin des instructions.....	3
III - Le Cycle d'Instruction et son implémentation.....	3
3.1. Phase FETCH (Recherche de l'instruction).....	4
3.2. Phase de DECODE (Décodage).....	4
3.3. Phase de EXECUTE (Exécution).....	5
3.4. Phase de MEMORY / WRITE-BACK (Accès mémoire et Écriture).....	5
IV - Implémentation du pipeline et gestion des aléas.....	5
4.1. La Gestion des Aléas de Structure (Structural Hazards).....	5
4.2. La Gestion des Aléas de Données (Data Hazards).....	5
V - Validation et Programme de Test.....	6
5.1. Le Code de Test.....	6
5.2. Analyse des Résultats du Simulateur.....	6
5.3. État Final du Processeur.....	7

I - INTRODUCTION : Objectifs et contexte pédagogique

1.1 Contexte pédagogique

L'informatique est aujourd'hui au cœur de presque tous les secteurs d'activité, allant de la médecine aux télécommunications. Au centre de ces technologies se trouve un élément fondamental : le microprocesseur (CPU). Ces puces électroniques sont les véritables cerveaux de nos machines et sont indispensables au bon fonctionnement de la majeure partie des outils informatiques. Comprendre leur architecture et leur fonctionnement interne est donc une étape incontournable pour tout informaticien souhaitant maîtriser l'interaction complexe entre le matériel (Hardware) et le logiciel(Software).



1.2 Objectif du projet

Dans le cadre de notre cursus en architecture des ordinateurs, nous avons conçu et implémenté un simulateur de processeur 16 bits. Ce projet a pour but d'observer en temps réel le comportement de l'architecture interne d'un CPU, notamment lors de l'exécution pipeliné des instructions. Les composants matériels que nous avons modélisés incluent : un banc de 4 registres, une mémoire de 32 cases, une Unité Arithmétique et Logique (UAL), ainsi qu'une unité de contrôle intégrant le Compteur Ordinal (PC) et le séquenceur. La simulation repose sur un jeu d'instructions réduit et ciblé : ADD, SUB, MOV, LOAD, STORE, JMP, NOP et HALT. Ce rapport s'articulera autour de trois axes majeurs : la présentation de l'architecture matérielle choisie ; l'implémentation du cycle d'exécution des instructions et l'ajout d'une architecture en pipeline (avec gestion des conflits) et enfin l'analyse des résultats obtenus lors des simulations.

II- ARCHITECTURE : Architecture et matériels

Afin de répondre aux exigences du projet tout en optimisant les performances de notre processeur 16 bits, nous avons opté pour une architecture matérielle de Von Neumann pipeliné.

Le modèle de Von Neumann a été privilégié face à l'architecture Harvard car Von Neumann permet de posséder qu'une seule mémoire.

Au sein de cette même mémoire, on retrouve à la fois les instructions du programme, les données manipulées, ainsi que leurs adresses. Ce choix simplifie la structure des bus, puisque le processeur interagit avec l'ensemble du système via un canal unique. Le processeur est ainsi composé de plusieurs unités fonctionnelles qui interagissent en synchronisation à chaque cycle d'horloge.

2.1. Les Registres

Les registres constituent la mémoire la plus rapide du processeur. Notre architecture intègre les éléments suivants :

Le banc de registres : le processeur dispose de 4 registres de travail, nommés R0, R1, R2 et R3. Ils stockent temporairement les données (sur 16 bits) en cours de traitement pour éviter des allers-retours coûteux avec la mémoire RAM.

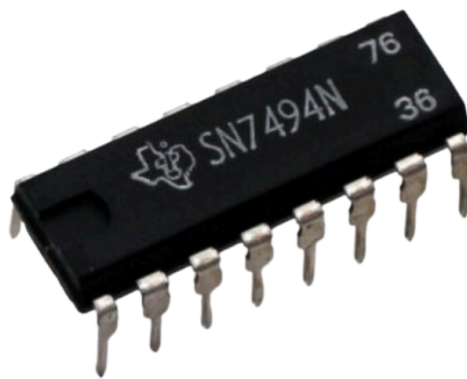
Le Compteur Ordinal (PC) : Ce registre pointe toujours vers l'adresse mémoire de la prochaine instruction à exécuter.

Les Registres Inter-étages (Pipeline) : L'architecture classique impose un unique Registre d'Instruction (IR). Cependant, pour supporter notre exécution en pipeline, nous avons fait évoluer ce concept en implémentant des registres de transition:

regD : Registre tampon dédié à l'étage de Décodage.

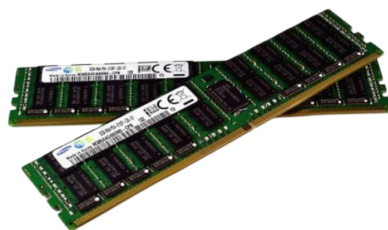
regE : Registre tampon dédié à l'étage d'Exécution.

regM : Registre tampon dédié à l'étage d'accès Mémoire.



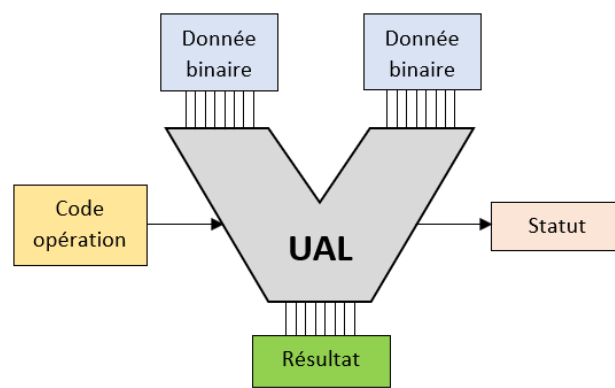
2.2. La Mémoire Centrale (RAM)

Le processeur interagit avec une mémoire principale permettant le stockage des instructions du programme et des données. Nous avons implémenté une mémoire de 32 cases (array); sa lecture et son écriture s'effectuent via des instructions dédiées (LOAD pour lire, STORE pour écrire).



2.3. L'Unité Arithmétique et Logique (UAL)

L'UAL est sollicitée lors de l'étage d'exécution. Elle prend en entrée les valeurs de deux registres sources (ou une valeur immédiate) et applique l'opération dictée par l'instruction en cours. Le résultat est ensuite redirigé vers le registre de destination.

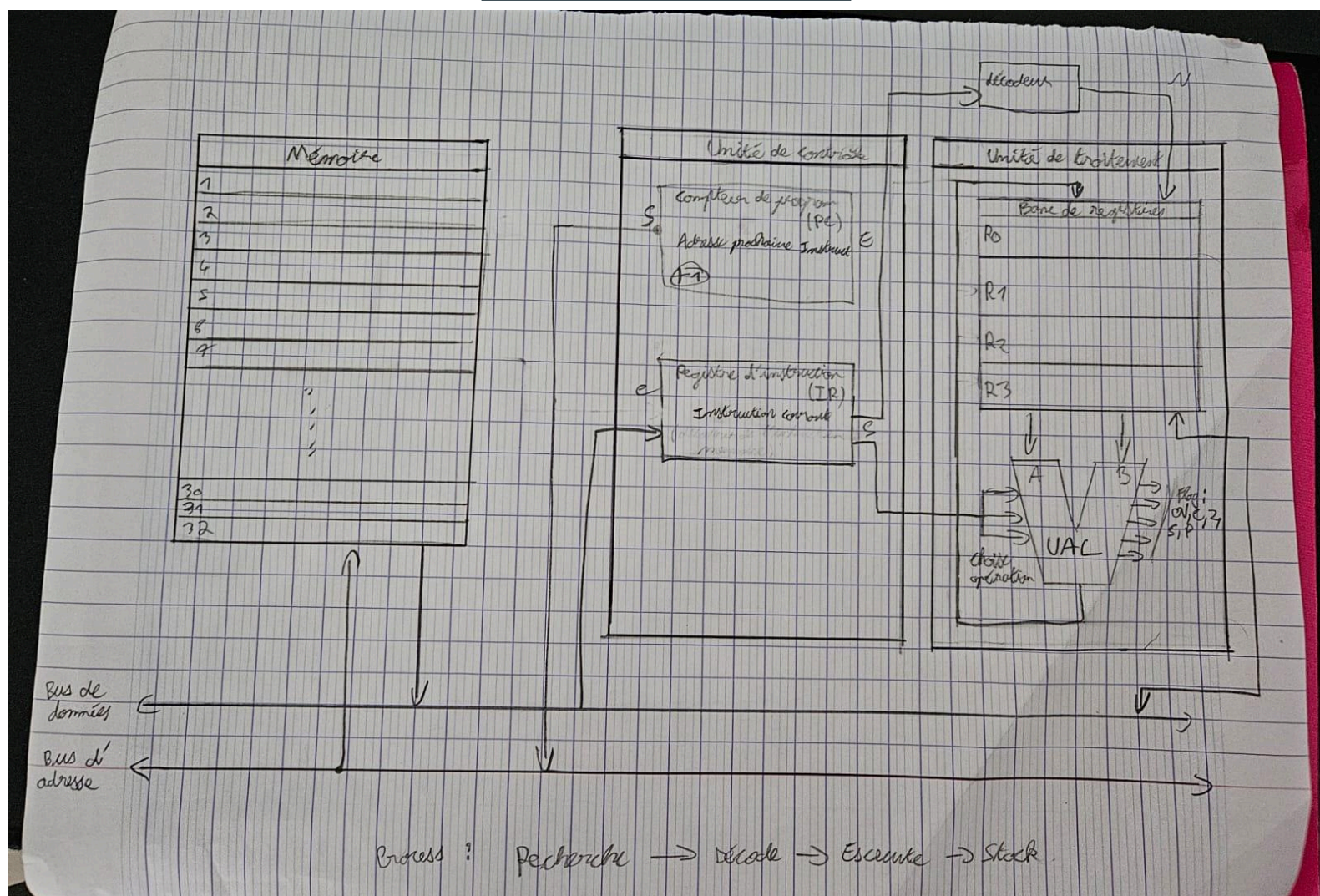


2.4. Le Chemin des instructions

C'est la particularité majeure de notre implémentation. Au lieu d'exécuter les instructions de manière séquentielle (une par une), notre processeur superpose les tâches grâce à un chemin de données découpé en 4 étages distincts :

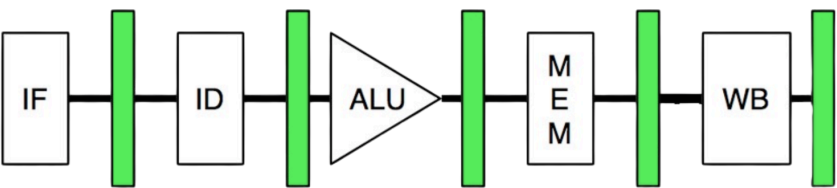
- **Fetch (Recherche)** : Le processeur lit l'instruction en mémoire à l'adresse indiquée par le PC.
- **Decode (Décodage)** : L'instruction brute est découpée pour en extraire le code opération (Opcode), les registres sources/destination et les éventuelles valeurs immédiates. C'est également ici que s'effectue la résolution des conflits de données (Data Hazards).
- **Execute (Exécution)** : L'UAL effectue le calcul ou le transfert de données.
- **Memory / Write-Back (Mémoire et Écriture)** : Les résultats sont inscrits définitivement dans le banc de registres ou stockés dans la mémoire RAM.

Schéma de notre architecture :



III - Le Cycle d'Instruction et son implémentation

Le cycle d'instruction représente la séquence d'opérations élémentaires qu'un processeur effectue pour exécuter une seule instruction machine. Afin d'optimiser les performances et de simuler un comportement réaliste, notre processeur ne résout pas une instruction d'un seul bloc. Nous l'avons découpée en quatre phases distinctes (qui correspondent aux étages de notre architecture en pipeline).



3.1. Phase FETCH (Recherche de l'instruction)

C'est le point de départ du cycle. L'objectif de cette phase est de récupérer l'instruction brute depuis la mémoire pour la ramener dans le processeur.

- **Lecture en mémoire** : Le processeur consulte le Compteur Ordinal (PC) pour connaître l'adresse de l'instruction en cours. Il va ensuite lire le mot binaire de 16 bits stocké à cette adresse dans la mémoire RAM.
- **Incrémentation** : Immédiatement après la lecture, le registre PC est incrémenté ($PC = PC + 1$) afin de pointer vers la prochaine instruction du programme.
- **Transmission** : L'instruction brute de 16 bits est placée dans le registre inter-étage pour être envoyée à la phase suivante.

3.2. Phase de DECODE (Décodage)

Cette phase agit comme le traducteur et l'aiguilleur du processeur. Elle prépare tout le terrain pour que le calcul puisse avoir lieu au cycle suivant.

- **Extraction des champs** : L'instruction brute de 16 bits est découpée à l'aide de décalages ($\gg$) et de masques binaires (ex: $\& 0xFF$). Le processeur identifie le code_op(l'action à réaliser), le registre de destination, et extrait les valeurs immédiates ou les numéros des registres sources.
- **Lecture des opérandes** : Le processeur va lire dans le banc de registres les valeurs actuelles des registres sources demandés par l'instruction.
- **Gestion des conflits** : C'est durant cette phase critique que notre processeur vérifie s'il y a un **Data Hazard**. Si une instruction a besoin d'une donnée qui est encore en cours de calcul dans les phases suivantes, et bien d'autres aléas de pipeline . La phase de décodage intercepte la donnée et permet de déterminer l'aléa courant et ainsi la traiter .

Tableau de décodage de l'instruction :

Exemple d'instruction : MOV R1, 5

MOV (4bits) [15 à 12]	R1 (2bits) [11 à 10]	R_destination (2 bits) absent [9 à 8]	5 (8 bits) [7 à 0]
0010	01	00	0000 0101

Explication du conflit DATA HAZARD:

ETAGES	FETCH	DECODE	EXECUTE	MEM/WB
CYCLE 1	MOV R1, 5			
CYCLE 2	ADD R3, R1, R2	MOV R1,5		
CYCLE 3	STORE R3, 10	ADD R3, R1, R2	MOV R1,5	
CYCLE 4	HALT	STORE R3, 10	ADD R3, R1, R2	MOV R1, 5

(STORE source, adresse destination) permet d'écrire en RAM et HALT prévenir l'arrêt imminent du programme.

STORE registre vers mémoire

LOAD mémoire vers registre

L'idée ici est que l'on applique l'idée du forwarding c'est à dire qu'au cycle 3, "ADD R3, R1, R2" ne connaît pas la valeur de r1 et comme l'instruction est seulement en train d'être décodé afin que l'UAL la traite, elle va venir attendre et prendre le résultat de l'exécution sur ce même cycle c'est à dire MOV R1,5 afin que R1 ait une valeur pour qu'au prochain cycle lorsque l'on a l'exécution il n'y ai pas de problème de valeur inconnu.

3.3. Phase de EXECUTE (Exécution)

C'est dans cette phase que le travail mathématique est réalisé, principalement par l'Unité Arithmétique et Logique (UAL). L'action dépend directement du code_op décodé lors de la phase précédente .

- **Opérations Arithmétiques (ADD, SUB) :** L'UAL prend les valeurs des deux registres sources, effectue le calcul, et génère le résultat.
- **Transfert (MOV) :** La valeur immédiate décodée est simplement laissée intacte pour être transférée au registre de destination.
- **Contrôle (JMP) :** Si l'instruction est un saut, c'est ici que la nouvelle adresse est injectée de force dans le Compteur Ordinal (PC), modifiant ainsi le flux du programme.

3.4. Phase de MEMORY / WRITE-BACK (Accès mémoire et Écriture)

C'est la dernière étape du cycle. L'instruction est terminée, il ne reste plus qu'à ranger le résultat à sa place définitive.

- **Accès à la RAM (LOAD et STORE) :** Si l'instruction nécessite d'interagir avec la mémoire principale, c'est ici que cela se produit. L'instruction **LOAD** va lire la case mémoire ciblée pour remplir le registre impliqué, tandis que **STORE** va inscrire la valeur du registre directement dans la RAM.
- **Écriture dans les registres (Write-Back) :** Pour toutes les autres instructions générant un résultat (**ADD, SUB, MOV, LOAD**), la valeur finale calculée ou lue est définitivement inscrite dans le banc de registres à l'adresse du registre de destination .
- **Nettoyage :** Une fois cette écriture terminée, l'instruction quitte le pipeline et est considérée comme totalement achevée et tout le système est remis à 0 pour un nouveau cycle d'instruction .

IV - Implémentation du pipeline et gestion des aléas

Pour passer d'une exécution séquentielle classique à une architecture parallèle, nous avons modélisé le pipeline en langage C en utilisant des structures de données dédiées pour chaque registre inter-étage (regD, regE, regM). Plutôt que de traiter une instruction de bout en bout, la boucle principale du simulateur cadence le processeur en appelant, à chaque cycle d'horloge, les quatre fonctions représentatives de nos étages dans l'ordre inverse : memoryWriteBack(), execute(), decode(), puis fetch().

Ce séquençage inversé est indispensable en programmation pour éviter qu'une instruction tout juste récupérée par le fetch n'écrase instantanément les données des étages suivants au cours du même cycle. Les informations (code_op, registres, valeurs calculées) sont ainsi stabilisées dans un étage, puis poussées vers le registre inter-étage suivant à la toute fin du cycle d'horloge. L'état global du CPU (le contenu de la mémoire RAM, le banc de registres et la position du pointeur PC) est mis à jour à chaque cycle, permettant de visualiser la progression simultanée de jusqu'à quatre instructions à des stades différents de leur cycle de vie.



Le défi majeur dans ce projet lors de l'implémentation du pipeline était la gestion des [aléas ou Hazard](#) . Notre simulateur gère deux grandes familles d'aléas : les aléas de structure et les aléas de données.

4.1. La Gestion des Aléas de Structure (Structural Hazards)

Un aléa de structure se produit lorsque le matériel ne peut pas supporter toutes les combinaisons d'instructions dans le même cycle d'horloge.

- **Le conflit d'accès à la mémoire :** Dans notre architecture, l'étage **Fetch** doit lire la mémoire RAM pour récupérer une nouvelle instruction. Au même moment, une instruction plus ancienne située à l'étage **Memory** peut avoir besoin de lire ou d'écrire une donnée en mémoire (via **LOAD ou STORE**) et étant donné que le processeur ne possède qu'un seul bus d'accès à la mémoire , ces deux étapes ne peuvent pas se faire en même temps.
- **La solution par l'insertion de bulles :** Pour résoudre ce conflit structurel, notre unité de contrôle détecte la présence simultanée d'un Fetch et d'un accès mémoire (LOAD/STORE). Elle va alors geler l'étage Fetch pendant un cycle en insérant une **bulle** (une instruction NOP). Cela décale l'instruction suivante et laisse la priorité à l'accès mémoire de l'instruction la plus ancienne.

4.2. La Gestion des Aléas de Données (Data Hazards)

Un aléa de données survient lorsqu'une instruction en phase de décodage dépend du résultat d'une instruction précédente qui n'a

pas encore validé son écriture.

- **Le problème de la dépendance** : exemple du programme de test demandé par le sujet :
 1. **MOV R1, 5** → Calcule la valeur 5 à l'étage Execute, mais l'écrira plus tard.
 2. **ADD R3, R1, R2** → L'instruction juste derrière veut lire R1 dès l'étage Decode .

Sans traitement, l'instruction ADD lirait une valeur obsolète dans R1 car l'écriture du MOV n'a pas encore eu lieu.

- **La solution par Forwarding** : Pour remédier à cet aléa et éviter de perdre en performance ; alors lors du décodage des instructions si on remarque une similitude entre un registre source de l'instruction courante et un registre destination de l'instruction précédente alors on va directement récupérer la valeur aux sorti de l' Execute de l'instruction précédente et l'initialiser lors du décodage dans le registre concerné .

V - Validation et Programme de Test

Pour valider le bon fonctionnement de notre simulateur et prouver la robustesse de notre architecture pipelinée, nous avons soumis le processeur au programme de test du projet.

5.1. Le Code de Test

Le programme exécuté est le suivant :

1. **MOV R1, 5** : Charge la valeur immédiate 5 dans le registre R1.
2. **MOV R2, 3** : Charge la valeur immédiate 3 dans le registre R2.
3. **ADD R3, R1, R2** : Additionne R1 et R2, et stocke le résultat (8) dans R3 (NB : Conflit de données potentiel avec les cycles précédents)
4. **STORE R3, [10]** : Écrit le contenu de R3 à l'adresse mémoire numéro 10.
5. **HALT** : Arrête proprement le processeur.

Dans une architecture séquentielle classique, ce programme prendrait 5 cycles complets (un par instruction). Dans notre modèle en pipeline à 4 étages, plusieurs instructions se superposent.

5.2. Analyse des Résultats du Simulateur

Grâce à l'affichage en console généré à chaque cycle d'horloge, nous avons pu observer la résolution en direct de cette situation :

```
PS C:\Users\kylia\OneDrive\Bureau\School\Cours\Architecture des ordinateurs\Projet\Codes.C> ./pipeline.exe
|-----Status-----|
|---Registres---|
R0 : 0000 R1 : 0000 R2 : 0000 R3 : 0000

|---UnitCommand---|
Program counter [PC] : 0000
Current instruction : 0000

|---Memory---|
[00]=3485 [01]=3883 [02]=1D02 [03]=438A [04]=F000 [05]=0000 [06]=0000 [07]=0000
[08]=0000 [09]=0000 [10]=0000 [11]=0000 [12]=0000 [13]=0000 [14]=0000 [15]=0000
[16]=0000 [17]=0000 [18]=0000 [19]=0000 [20]=0000 [21]=0000 [22]=0000 [23]=0000
[24]=0000 [25]=0000 [26]=0000 [27]=0000 [28]=0000 [29]=0000 [30]=0000 [31]=0000

|-----Status-----|
|---Registres---|
R0 : 0000 R1 : 0000 R2 : 0000 R3 : 0000

|---UnitCommand---|
Program counter [PC] : 0001
Current instruction : 3485

|---Memory---|
[00]=3485 [01]=3883 [02]=1D02 [03]=438A [04]=F000 [05]=0000 [06]=0000 [07]=0000
[08]=0000 [09]=0000 [10]=0000 [11]=0000 [12]=0000 [13]=0000 [14]=0000 [15]=0000
[16]=0000 [17]=0000 [18]=0000 [19]=0000 [20]=0000 [21]=0000 [22]=0000 [23]=0000
[24]=0000 [25]=0000 [26]=0000 [27]=0000 [28]=0000 [29]=0000 [30]=0000 [31]=0000

|-----Status-----|
|---Registres---|
R0 : 0000 R1 : 0000 R2 : 0000 R3 : 0000

|---UnitCommand---|
Program counter [PC] : 0002
Current instruction : 3883

|---Memory---|
[00]=3485 [01]=3883 [02]=1D02 [03]=438A [04]=F000 [05]=0000 [06]=0000 [07]=0000
[08]=0000 [09]=0000 [10]=0000 [11]=0000 [12]=0000 [13]=0000 [14]=0000 [15]=0000
[16]=0000 [17]=0000 [18]=0000 [19]=0000 [20]=0000 [21]=0000 [22]=0000 [23]=0000
[24]=0000 [25]=0000 [26]=0000 [27]=0000 [28]=0000 [29]=0000 [30]=0000 [31]=0000
```

```

PS C:\Users\kylia\OneDrive\Bureau\School\Cours\Architecture des ordinateurs\Projet\Codes.C> ./pipeline.exe
|-----Status-----|
|---Registres---|
R0 : 0000 R1 : 0000 R2 : 0000 R3 : 0000

|---UnitCommand---|
Program counter [PC] : 0003
Current instruction : 1D02

|---Memory---|
[00]=3485 [01]=3883 [02]=1D02 [03]=438A [04]=F000 [05]=0000 [06]=0000 [07]=0000
[08]=0000 [09]=0000 [10]=0000 [11]=0000 [12]=0000 [13]=0000 [14]=0000 [15]=0000
[16]=0000 [17]=0000 [18]=0000 [19]=0000 [20]=0000 [21]=0000 [22]=0000 [23]=0000
[24]=0000 [25]=0000 [26]=0000 [27]=0000 [28]=0000 [29]=0000 [30]=0000 [31]=0000

|-----Status-----|
|---Registres---|
R0 : 0000 R1 : 0005 R2 : 0003 R3 : 0000

|---UnitCommand---|
Program counter [PC] : 0004
Current instruction : 438A

|---Memory---|
[00]=3485 [01]=3883 [02]=1D02 [03]=438A [04]=F000 [05]=0000 [06]=0000 [07]=0000
[08]=0000 [09]=0000 [10]=0000 [11]=0000 [12]=0000 [13]=0000 [14]=0000 [15]=0000
[16]=0000 [17]=0000 [18]=0000 [19]=0000 [20]=0000 [21]=0000 [22]=0000 [23]=0000
[24]=0000 [25]=0000 [26]=0000 [27]=0000 [28]=0000 [29]=0000 [30]=0000 [31]=0000

|-----Status-----|
|---Registres---|
R0 : 0000 R1 : 0005 R2 : 0003 R3 : 0000

|---UnitCommand---|
Program counter [PC] : 0005
Current instruction : F000

|---Memory---|
[00]=3485 [01]=3883 [02]=1D02 [03]=438A [04]=F000 [05]=0000 [06]=0000 [07]=0000
[08]=0000 [09]=0000 [10]=0000 [11]=0000 [12]=0000 [13]=0000 [14]=0000 [15]=0000
[16]=0000 [17]=0000 [18]=0000 [19]=0000 [20]=0000 [21]=0000 [22]=0000 [23]=0000
[24]=0000 [25]=0000 [26]=0000 [27]=0000 [28]=0000 [29]=0000 [30]=0000 [31]=0000

```

```

|-----Status-----|
|---Registres---|
R0 : 0000 R1 : 0005 R2 : 0003 R3 : 0008

|---UnitCommand---|
Program counter [PC] : 0006
Current instruction : 0000

|---Memory---|
[00]=3485 [01]=3883 [02]=1D02 [03]=438A [04]=F000 [05]=0000 [06]=0000 [07]=0000
[08]=0000 [09]=0000 [10]=0008 [11]=0000 [12]=0000 [13]=0000 [14]=0000 [15]=0000
[16]=0000 [17]=0000 [18]=0000 [19]=0000 [20]=0000 [21]=0000 [22]=0000 [23]=0000
[24]=0000 [25]=0000 [26]=0000 [27]=0000 [28]=0000 [29]=0000 [30]=0000 [31]=0000

|-----Status-----|
|---Registres---|
R0 : 0000 R1 : 0005 R2 : 0003 R3 : 0008

|---UnitCommand---|
Program counter [PC] : 0007
Current instruction : 0000

|---Memory---|
[00]=3485 [01]=3883 [02]=1D02 [03]=438A [04]=F000 [05]=0000 [06]=0000 [07]=0000
[08]=0000 [09]=0000 [10]=0008 [11]=0000 [12]=0000 [13]=0000 [14]=0000 [15]=0000
[16]=0000 [17]=0000 [18]=0000 [19]=0000 [20]=0000 [21]=0000 [22]=0000 [23]=0000
[24]=0000 [25]=0000 [26]=0000 [27]=0000 [28]=0000 [29]=0000 [30]=0000 [31]=0000

```

5.3. État Final du Processeur

À la fin de la simulation (lorsque l'instruction HALT est décodée et vide le pipeline), le simulateur affiche l'état mémoire et machine final permettant de valider mathématiquement notre code :

- **Le banc de registres** : Le registre R1 contient 5, R2 contient 3 et R3 contient bien 8.
- **La mémoire RAM** : L'inspection de la case mémoire numéro 10 montre qu'elle stocke désormais la valeur 8.

